

Unit-3.

I/O Interfacing!~

Memory and I/O interfacing with 8086!

*. microprocessor - don't have in-built memory.

Memory - group of Register.

Reg - group of flipflop.

flipflop - store 1 bit data.

It is simple device that can be used to store information.

↑
Memory.

ROM

RAM

⇒ Non-Volatile.

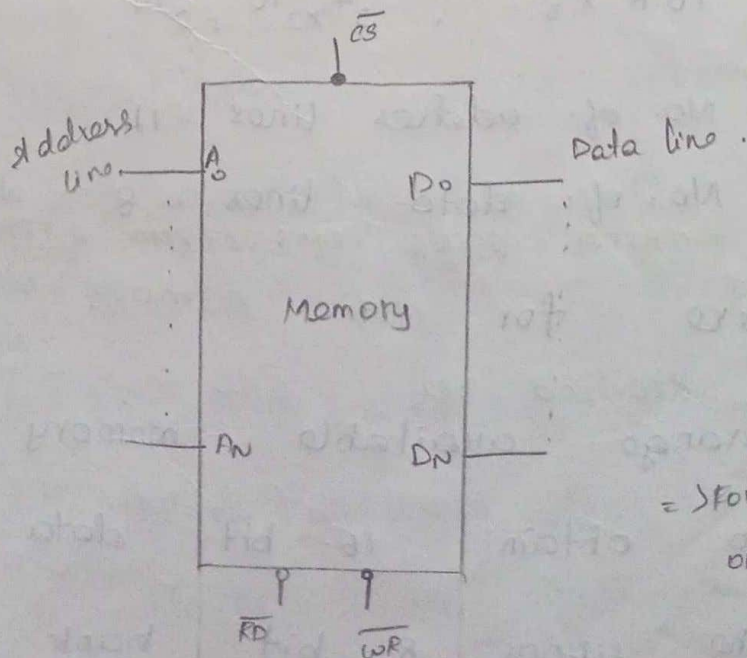
⇒ Volatile.

⇒ store permanent data.

⇒ store temporary data

(EPROM, PROM, EEPROM).

Schematic memory:



⇒ For ROM
OE instead of
RD / $\overline{WR}$

* It has address line in order to issue address.

* address line depends on (memory size)

* We should have 'N' no. of Data lines.

* we have other control signal is chip select signal (it should be active low) $[\overline{CS}]$ to select this chip

For, $(1K \times 8)$ chip, there will $2^{10} = 1024$ location with byte in each location.
 (no. of data lines $\rightarrow$ address line)

$$2K \times 8 = 2KB \text{ (memory capacity)}$$

1024 $\times$ 8 = memory location.

For, $4K \times 8$, $2^2 \times 2^{10} = 2^{12} \rightarrow$ address line

For, $16K \times 8$, $2^4 \times 2^{10} = 2^{14}$

No. of address lines - 14,

No. of data lines - 8

MEMORY INTERFACE USING RAMS, EPROM, EEPROMS:

Procedure for static memory interfacing with

8086 is described as

① Arrange available memory chips so as to obtain 16-bit data bus width.

* The upper 8 bit bank is called odd memory bank, the lower 8 bit bank

is called even memory bank.

② connect available address memory address lines of memory chips with those of microprocessor & also connect the ^{memory} read / write ¹⁶ to processor control signals.

* Connect the 16 bit data bus (lines) of memory bank with that of 8086 data bus.

③ The remaining address lines of microprocessor, $\overline{BHE}$ & A_0 are used for decoding the required chip select signals for the odd & even memory bank.

* CS of memory is derived from o/p of decoding circuit.

④ Interface two $4K \times 8$ EPROM and two $4K \times 8$ RAM chips with 8086, select suitable map.

Sol:

two $4K \times 8$ EPROM = $8K \times 8$ EPROM

No. of address lines = 13 $(2^3 \times 2^{10} = 2^{13})$

No. of data lines = 8

two $4K \times 8$ RAM = $8K \times 8$ RAM

No. of address lines = 13

No. of Data lines = 8

A_{13}	A_0	$\overline{BHE}$
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

A_0 is used for CS signal derivation.

If $A_0 = 0 \rightarrow$ we have to access RAM.

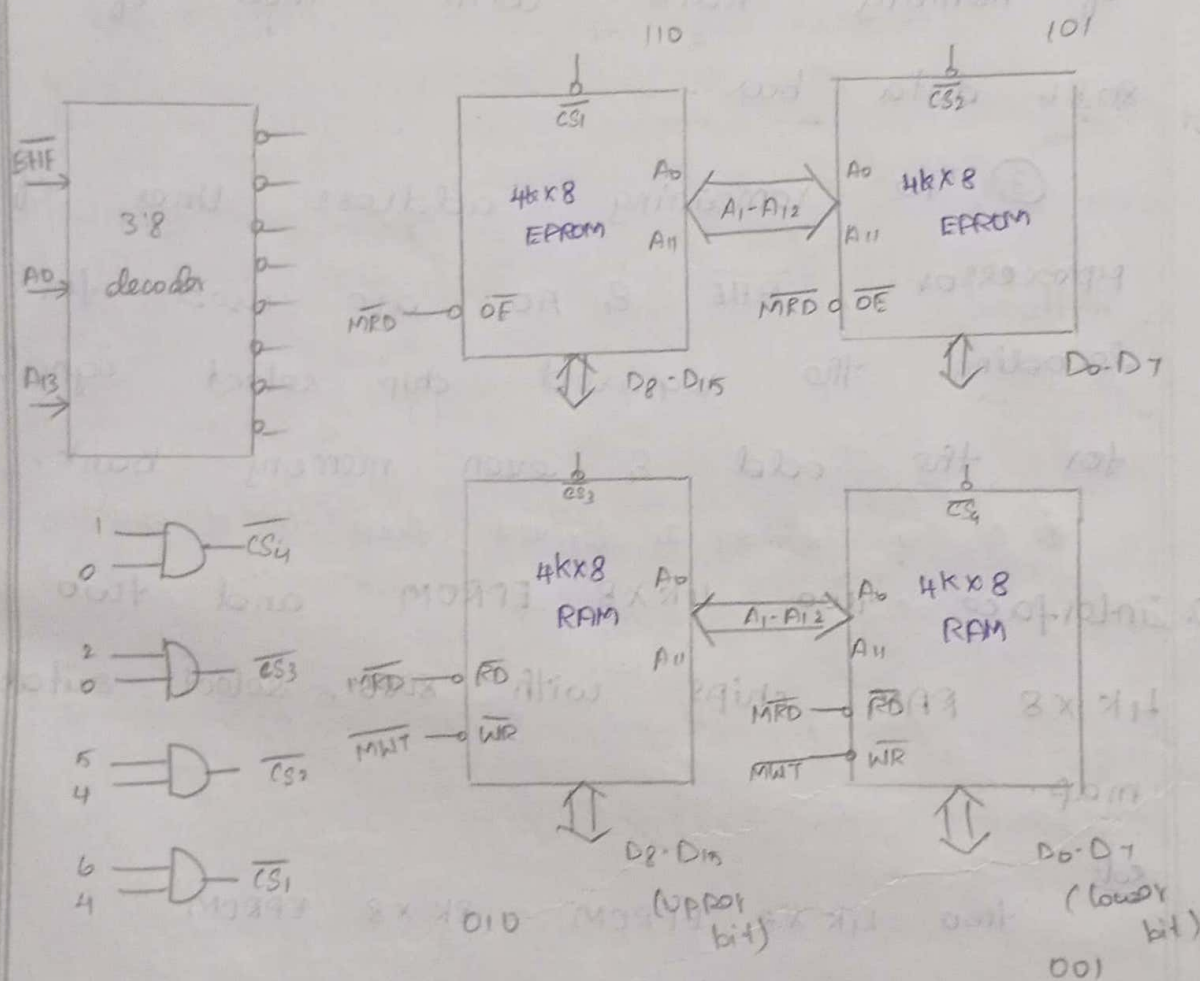
If $A_0 = 1 \rightarrow$ we have to access EPROM.

A_0 is '0' for even (lower bit)

If BHE is high access only (lower bit)

If BHE is low (0) access both (higher & lower bit)

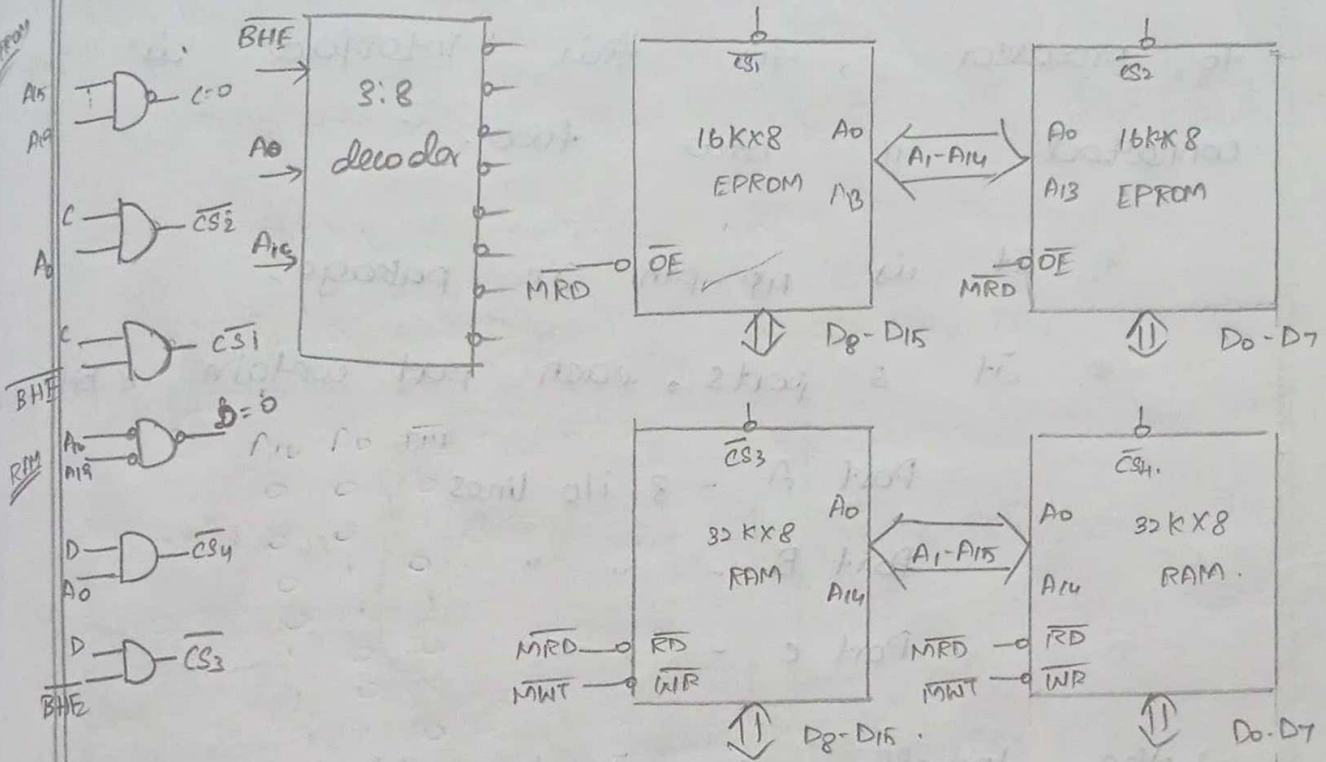
To derive the chip select signal decoder is used.



	A_9	A_{18}	A_{17}	A_{16}	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
FFFFH	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
FF00H	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
FDFFFH	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
FC00H	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

unused registers additional used.

② Design a Interface between 8086 CPU &
Two chips of 16K X 8 EPROM and two chips of 32 K X 8 RAM, select starting address of EPROM suitable, RAM address must start at 00000 H.

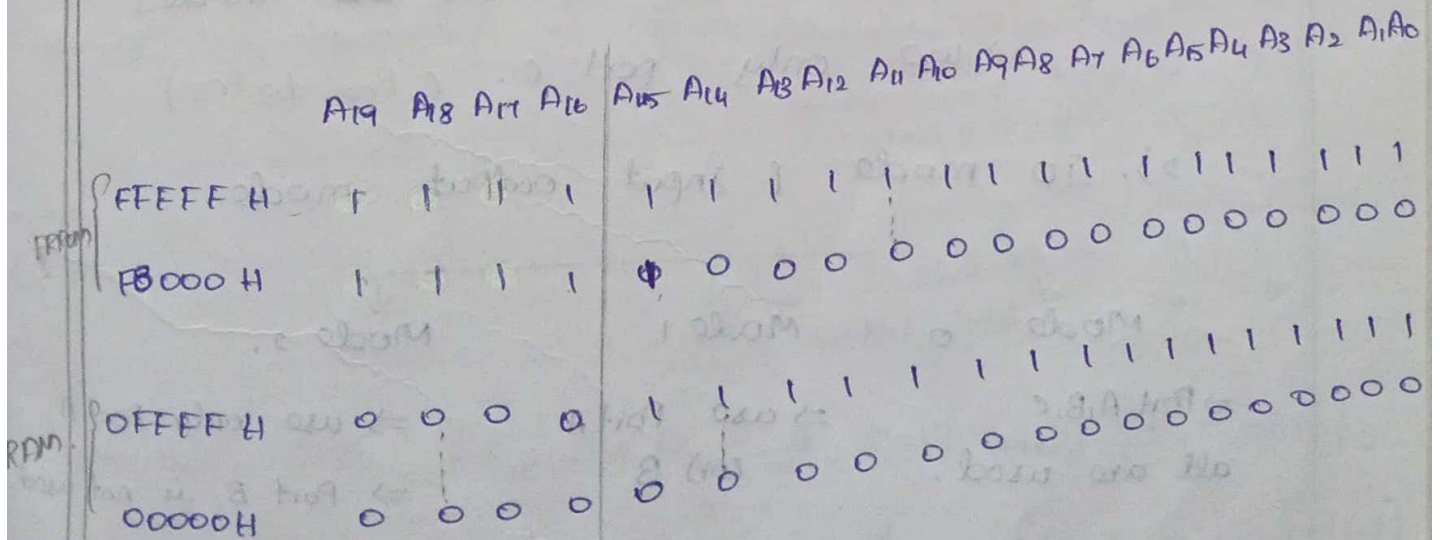


→ 2 16K X 8 EPROM.

Address lines = $2^{14} \times 2^1 = 15$
data lines = 8

→ 2 32K X 8 RAM.

Address lines = 16
data lines = 8.



PROGRAMMABLE PERIPHERAL INTERFACE 8255.

* also called as parallel communication.

* we can't connect directly peripheral to processor, for this interface is connected in btw two.

* It is 40-pin IC package.

* It has 3 ports, each port contains 8 bit wide

Port A - 8 i/o lines	} 24 i/o lines.
Port B - " "	
Port C - " "	

⇒ the totally 24 input output lines.

* It can be configured in two modes.

1). BSR mode [Bit set (or) Reset mode]

⇒ we use only port C. (Pc0 to Pc7)

2). IO mode (input output mode).

Mode 0

⇒ Port A, B, C all are used.

Mode 1

⇒ use Port A (or) B

⇒ Port C provides handshaking signals.

Mode 2

⇒ use only Port A

⇒ Port B is not used.

⇒ Port C provides handshaking signals.

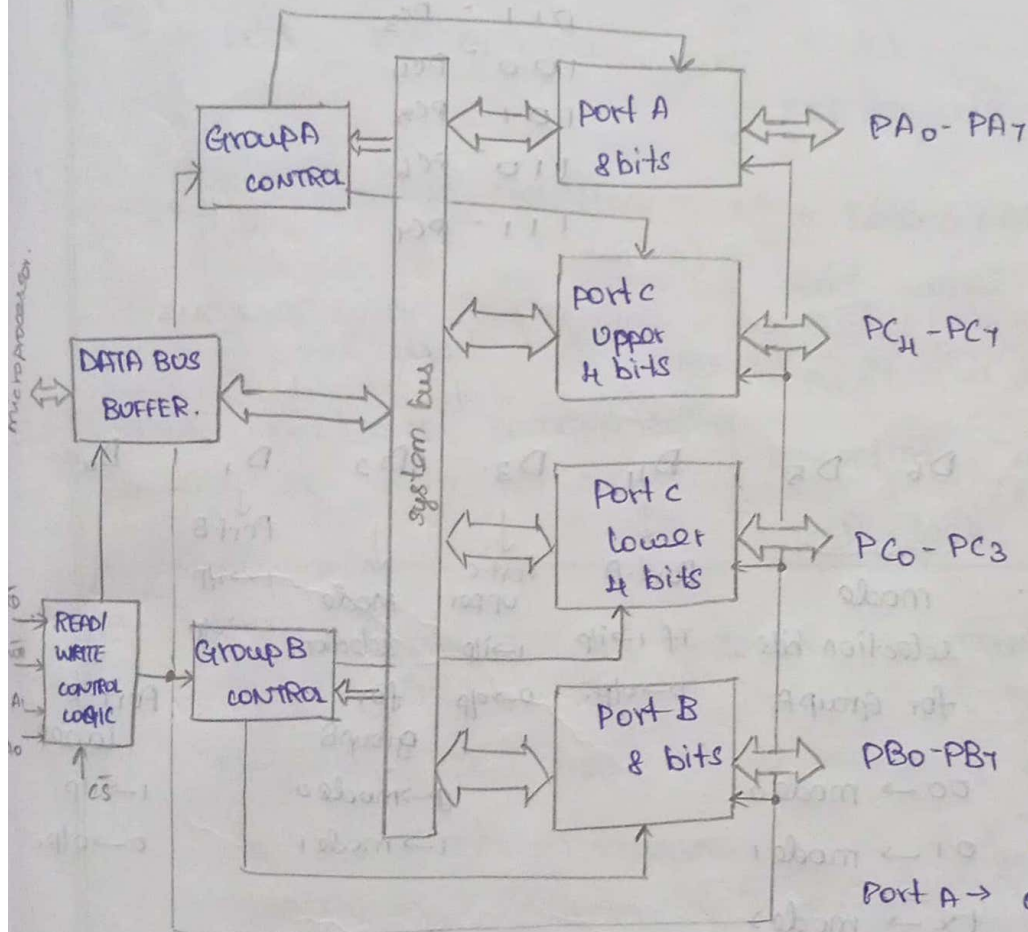
ARCHITECTURE OF 8255.

Port - c lower = PC_0 to PC_3 (4 bit)

Port c upper = PC_4 to PC_7 (4 bit)

we can use individual bit (or) 4 bit in Port - c

Port - A & Port - B used in 8 bits only.



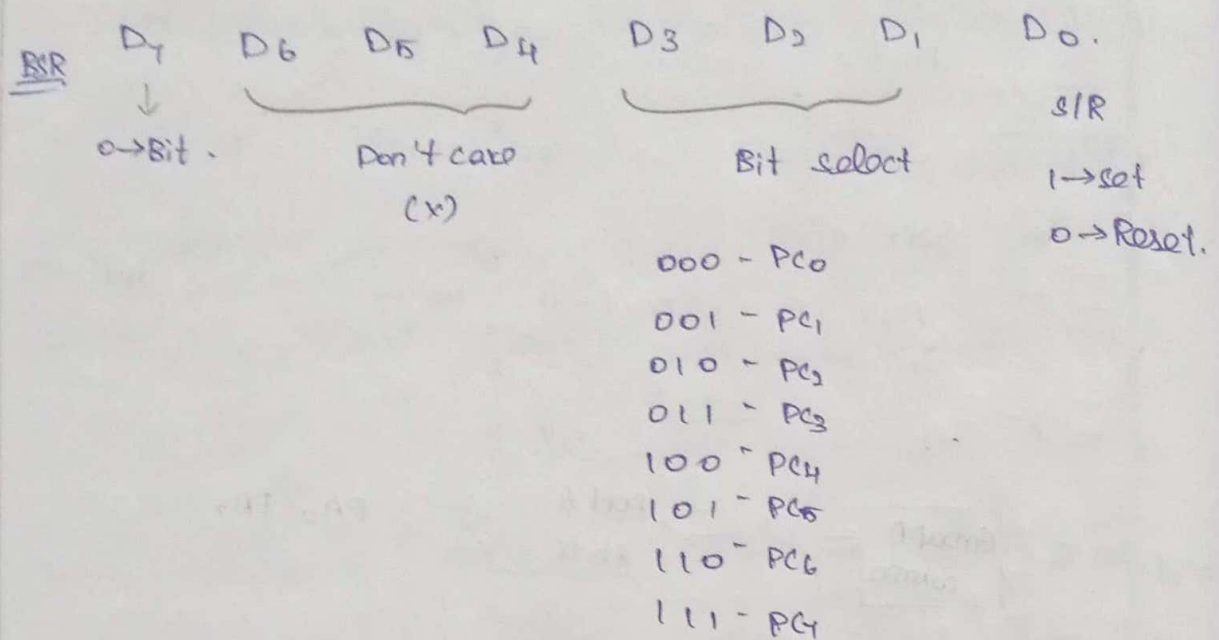
	$\overline{CS}$	A_1	A_0
Port A $\rightarrow$	0	0	0
Port B $\rightarrow$	0	0	1
Port c $\rightarrow$	0	1	0
CWR $\rightarrow$	0	1	1
Not selected $\rightarrow$	1	x	x

For BSR [D_6, D_5, D_4 - is Don't care], [D_7 - zero],

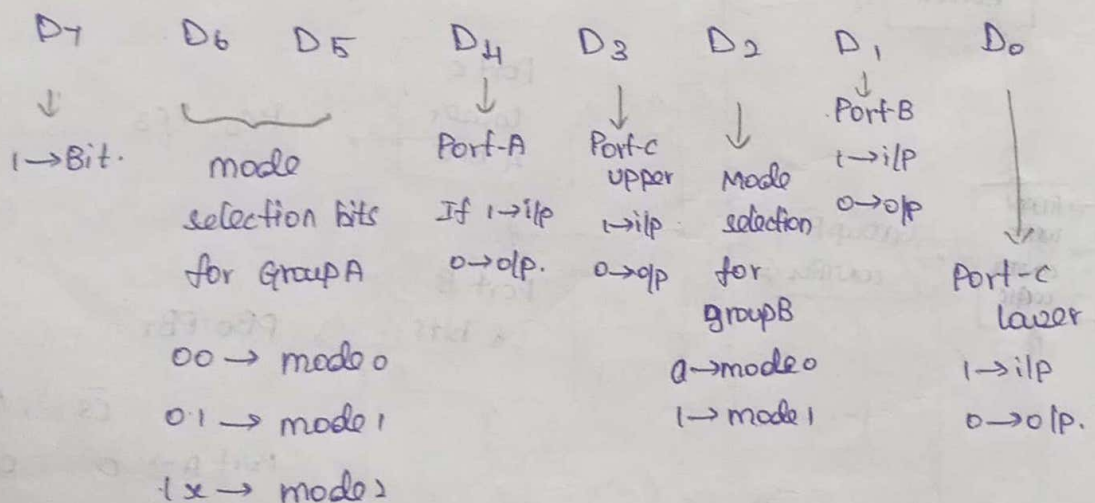
CWR $\rightarrow$ control write Read. [D_3, D_2, D_1 - selected]

1 $\rightarrow$ set, 0 $\rightarrow$ Reset.

CWR. LSB $\rightarrow$ set (or) Reset.



IO!



eg:

If mode = 1, Port A $\rightarrow$ i/p, Port B, C $\rightarrow$ o/p, what is control word for IO.

(1 0 1 1 0 0 0 0) = B0 H. H $\rightarrow$ hexadecimal.
 B $\rightarrow$ 1011
 0 $\rightarrow$ 0000

Parallel communication	Serial communication.
------------------------	-----------------------

* Multiple bits transmitted over multiple channel	* single bit transmit over single channel.
---	--

* speed high	
--------------	--

	* speed low.
--	--------------

* less preferable.	
--------------------	--

	* It is more preferable.
--	--------------------------

* Expensive (multiple wires)	
------------------------------	--

	* Low cost (single wire) and more reliable.
--	---

* It is used for only short distance communication connection b/w within the device.	
--	--

	* It is used for connecting device to device (or) long distance communication.
--	--

3 modes of serial communication

1. Simplex - It has 1 channel & [Receive & (one way commu) transmission can't occur simultaneously]

2. Half duplex - 2 way communication possible. Not simultaneously.

3. full duplex - 2 way communication, Each transmission & reception has separate channel so occur simultaneously

Another classification of serial communication.

1. Synchronous - both transmission & reception

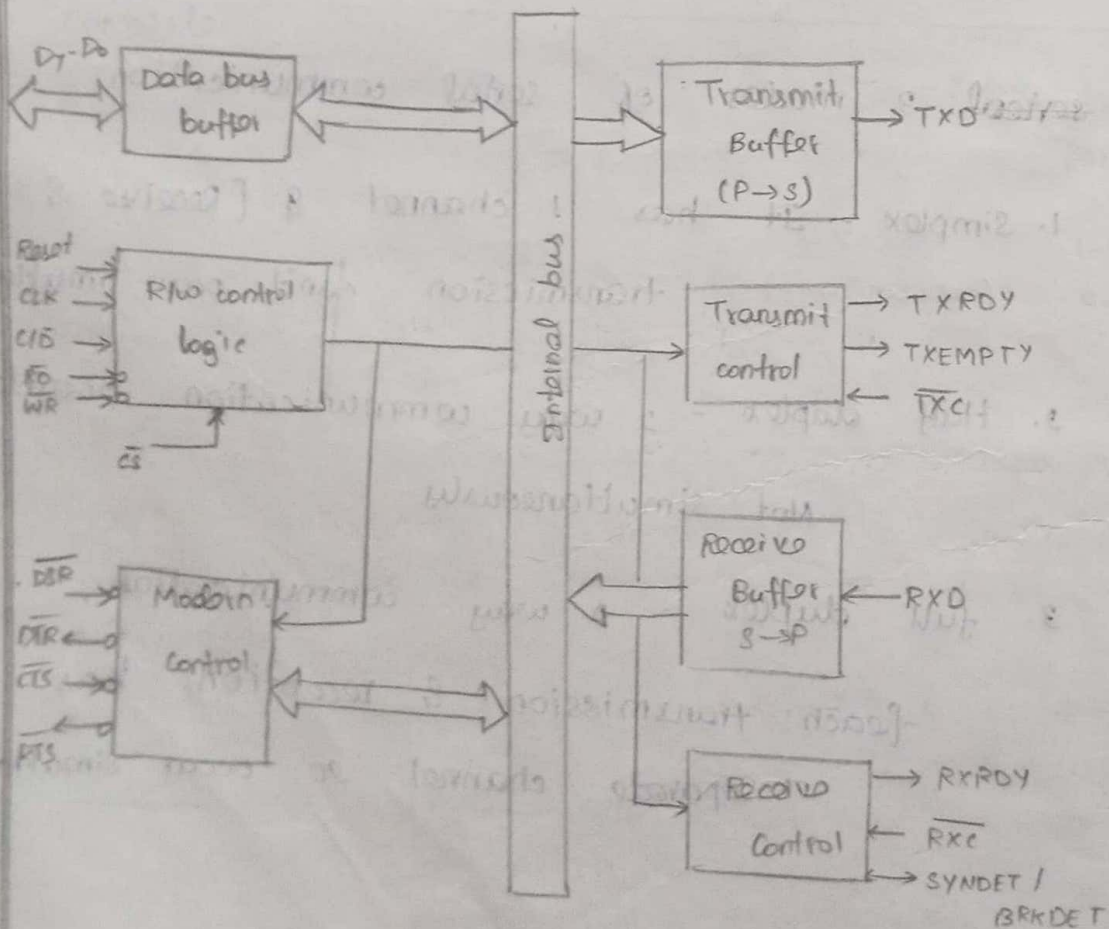
occur at same speed (probe is shared by both)

2. Asynchronous - They both not at same speed becoz both use separate probe.

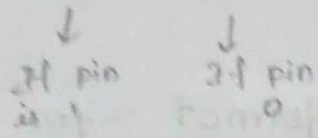
key component: (It has ^{no} id to serial & serial to ^{no} id converter)

USART - Universal ^{no} synchronous Asynchronous
Receive transmit.

ARCHITECTURE of USART:



C/D $\rightarrow$ command / data. (from μ controller it receives whether C/D)



TXRDY $\rightarrow$ transmits Ready. (It tells, it is ready) ready to accept the new character which is to be transmitted

TXD $\rightarrow$ transmit Data pin.

TXEMPTY $\rightarrow$ indication of No characters pending which is to be transmitted. Now the buffer is empty

(from 8251 to μ controller)

$\hookrightarrow$ both TXRDY & TXEMPTY.

$\overline{\text{TXE}}$ - transmit clock.

RXD $\rightarrow$ receive data pin, (receives serial data convert to 11el) goes to Data bus buffers.

RXRDY $\rightarrow$ Indication given by 8251 that it is ready to accept.

SYNDET $\rightarrow$ Synchronous detect. (Break detect)

BRKDET

for Receiver (from μ controller or External to 8251)

$\overline{\text{DTR}}$ - Data Terminal Ready.

CTS - clear to send

$\overline{\text{RTS}}$ - Request to send.

DSR - Data Set Ready.

Three Instructions:

1. Mode Instruction format for asynchronous.

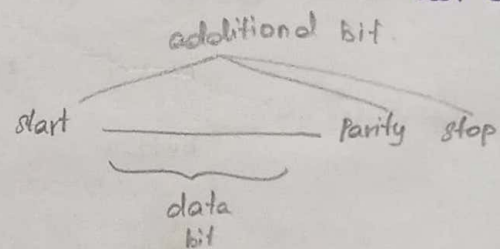
* To detect & correct error we use parity bit.

* while transmitted the data bit has additional bits; start, Parity, stop bit.

* while receiving additional bits are removed.
* It is 8-bit.

S ₂	S ₁		EP	(Parity enable)
			PEN	
stop bit selection.			↓	↓
0	0	- Invalid	1 → even parity	1 → parity required
0	1	- 1 bit		
1	0	- 1 1/2 bit	0 → odd parity.	0 → parity not required
1	1	- 2 bit		

L ₂	L ₁	⇒ length of the character.
0	0	- 5 bits
0	1	- 6 bits
1	0	- 7 bits
1	1	- 8 bits.



B ₂	B ₁	
Baud rate select.		
0	0	→ synchronous mode.
0	1	→ 1x asynchronous mode.
1	0	→ 16x " "
1	1	→ 64x " "

Mode Instruction

format for "Synchronous."
(Get Immediate Response)

scs
↳ single character synchronous.

ESD
↳ External

1 → single synchronous character.

synchronous Detector.

0 → double " "

EP, PEN, L₂, L₁ → are same as asynchronous.

B₂ B₁ → 0 0.

asynchronous:

B ₂	B ₁	EP	PEN	L ₂	L ₁	B ₂	B ₁
----------------	----------------	----	-----	----------------	----------------	----------------	----------------

Synchronous:

SCS	ESD	EP	PEN	L ₂	L ₁	0	0
-----	-----	----	-----	----------------	----------------	---	---

2. Command Instruction format:

EN.

IR - Internal Reset

1 → hunt for
synchronous
character.

RTS - Request to send.

EP - Error Reset

SBRK - send break character.

RXE - Receive enable.

DTR - Data terminal Ready.

TXEN - transmit enable.

EN	IR	RTS	EP	SBRK	RXE	DTR	TXEN
----	----	-----	----	------	-----	-----	------

3. Status Information:

SYNCDDET → synchronous detector.

TXEMPTY → all are sent.

FE → error.

DSR	SYNCDDET	FE	OE	PE	TXEMPTY	RXRDY	TXRDY
-----	----------	----	----	----	---------	-------	-------

ADC Interfacing with 8086:

$\overline{CS}$	1	30	VCC
$\overline{RD}$	2	19	clk out
$\overline{WR}$	3	18	DB0 (LSB)
clk in	4	17	DB1
$\overline{INTR}$	5	16	DB2
V IN +	6	15	DB3
V IN -	7	14	DB4
analog gnd	8	13	DB5
Vref/2	9	12	DB6
Dig Gnd	10	11	DB7 (MSB)

IN +, IN - $\rightarrow$ analog i/p.

features:

- * 8 bit successive approximation ADC.
 - * Conversion time - 100 μ sec.
 - * access time - 135 μ sec.
 - * It has on chip clock generator.
 - * Works with 5V power supply.
 - * It does not require any zero adjustment.
- $\Rightarrow$ To start of the conversion is indicated by two signal.

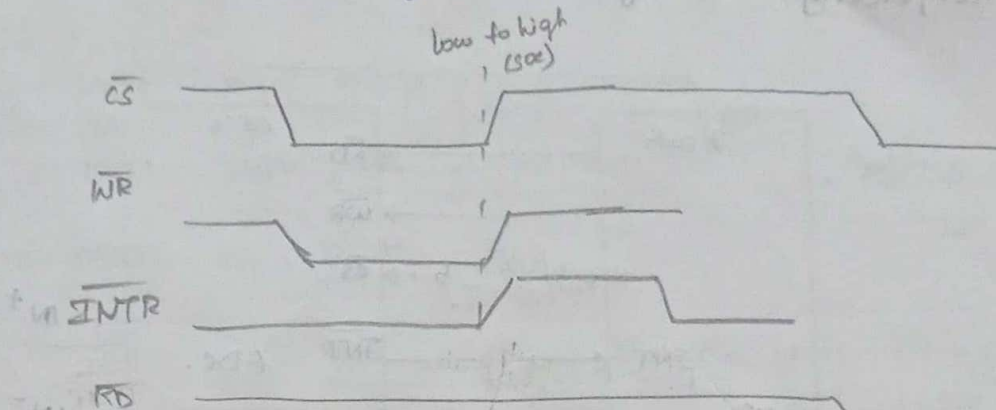
1. $\overline{CS}$

2. $\overline{WR}$

low to high transition of these

two signals.

soc $\rightarrow$ start of conversion.



(before issuing soc signal we should ensure that Analog signal is Stable)

1. Issue of soc signal.

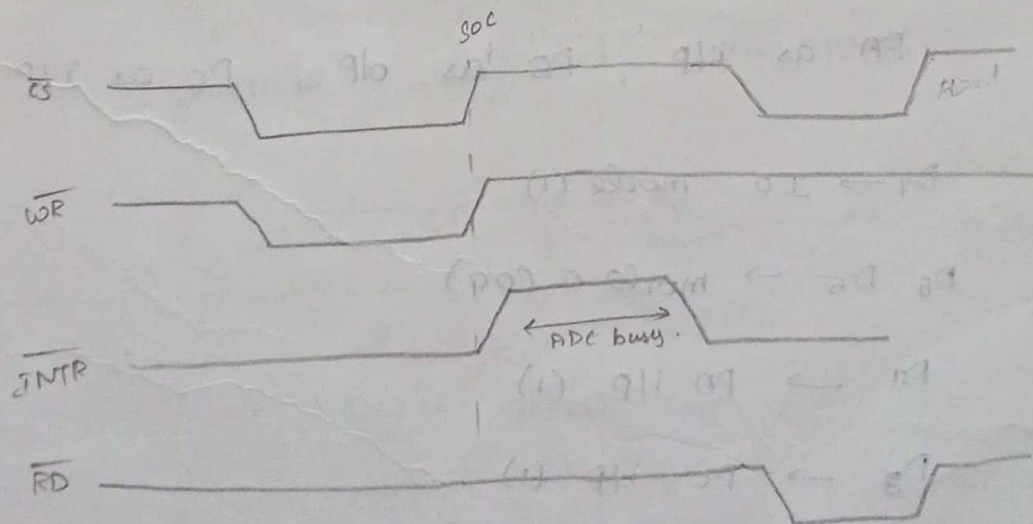
$\Rightarrow$ INTR pin go high, indicate ADC is busy, in doing the conversion process.

$\Rightarrow$ End of conversion - Indicated by INTR pin it goes low.

2. Monitor the INTR pin.

To Read : CS, RD pins go to [from low to high].

3. Give signal to Read signal.



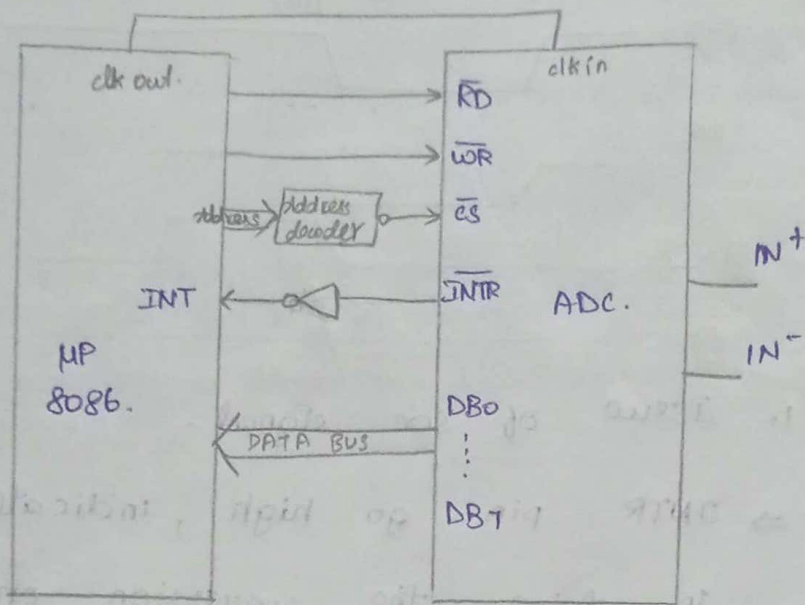
(Data) is Read.

(Data) is Read.

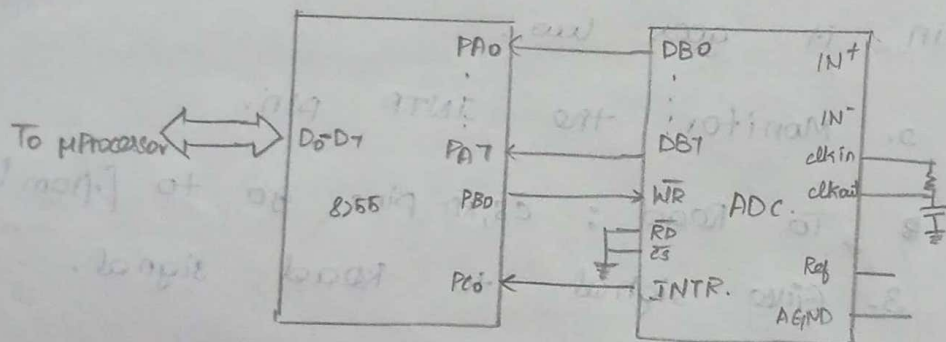
(Data) is Read.

(Data) is Read.

Interfacing diagram:



Program:



PA as i/p, PB as o/p, PC as I/P

D7 → IO mode (1)

D6 D5 → mode 0 (00)

D4 → PA i/p (1)

D3 → PC i/p (1)

D2 → mode 0 (0)

D1 → PB o/p (0)

D0 → PC lower o/p (0).

$$\underline{10011000} = 98H.$$

CWR → peripheral device.

```
MOV AL, 98H } control word
OUT CWR, AL  } eg is initialized
```

In Port B place use addresses.

```
MOV AL, FFH }
OUT PortB, AL } high to low transition.
MOV AL, FCH }
OUT PortB, AL }
MOV AL, FFH }
OUT PortB, AL }
```

1111 1111 FF (high)
1111 1100 FC (low)
1111 1111 FF (high)
create soc.

INTR → busy - 1
ready - 0.

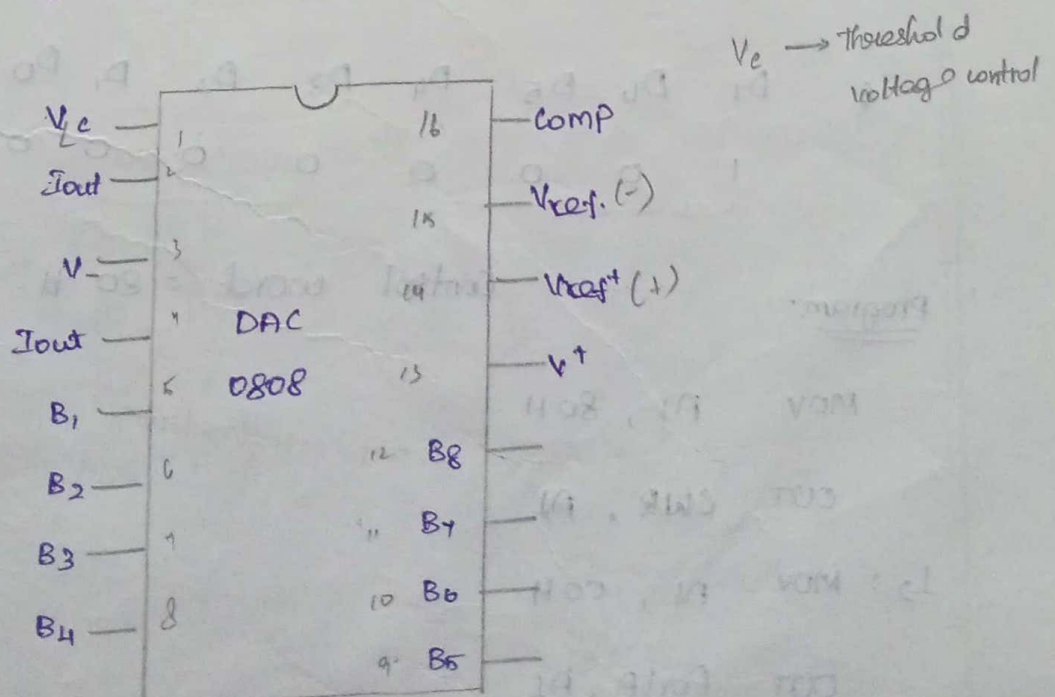
INTR monitor.
LI: IN AL, PortC
AND AL, 01H → to check interrupt is 0 (or) 1.
JNZ LI.

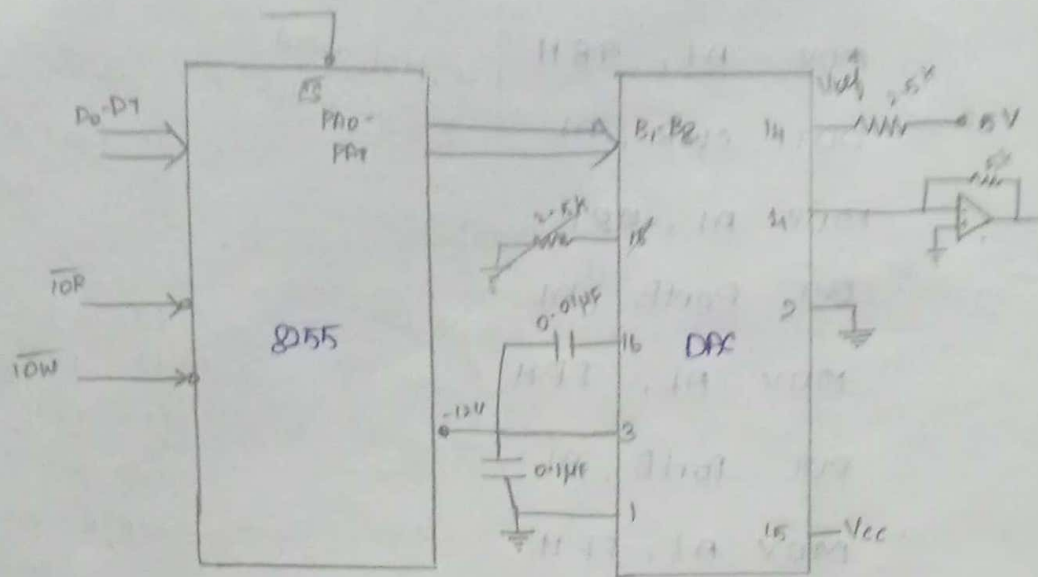
Read. ← IN AL, PortA

[If INTR 0, AND=0
So come out of this loop]

DAC Interfacing With 8086.

To generate different waveform:





In DAC, signals are come from μ Process.
 so no need to initialize any port as i/p,
 we use as o/p

1000 0000 - 80H.
 D7 D6 D5 D4 D3 D2 D1 D0.

Generate Square Waveform:

use 210, mode 0, all ports are o/p.

	D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	0	0	0	0	0	0

Program: Control word = 80H.

```

MOV AL, 80H
OUT CNTR, AL
} 8255 Initialization.
L2: MOV AL, 00H
OUT PortA, AL
CALL DELAY
MOV AL, FFH
CALL DELAY
JMP L2
  
```


DELAY : MOV CL, FFH

L1: DEC CL

JNZ L1

RET.

Generate triangular waveform.

MOV AL, 80H

OUT CWR, AL

MOV AL, 00H

L1: OUT PORTA, AL

INC AL

CMP AL, FFH

JB L1

~~MOV AL, FFH~~

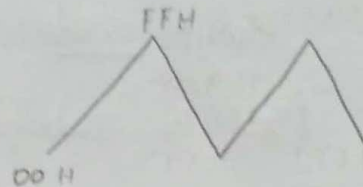
L2: OUT PORTA, AL

DEC AL

CMP AL, 00H

JA L2

SJMP L1.



CWR = C6

PORTA = C0.

Generate Ramp (Sawtooth) waveform.

MOV AL, 80H

OUT CWR, AL

MOV AL, 00H

L1: OUT PORTA, AL

INC AL

CMP AL, FFH

JB L1

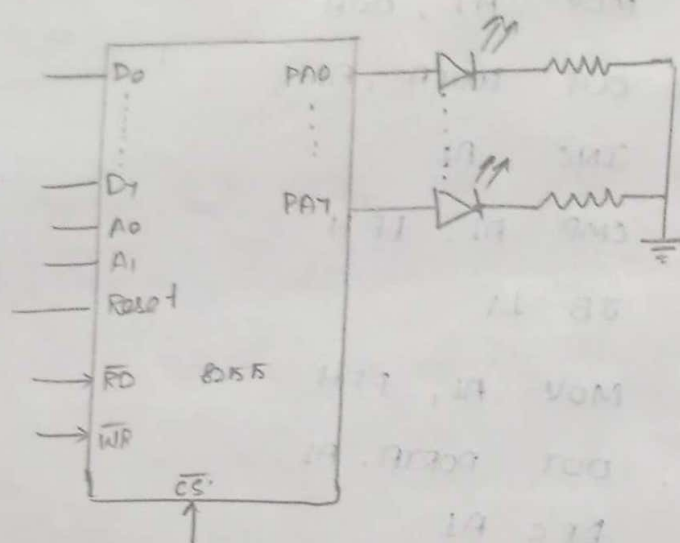
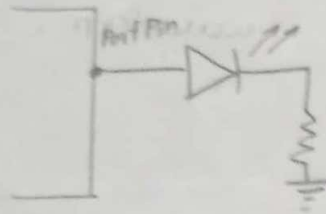
MOV AL, 00H

SJMP L1

LED INTERFACING WITH 8086

LED turn ON, when $I/P = 0$

LED turn OFF, when $I/P = 1$.



Program:

to turn ON & OFF alternatively,

```

MOV AL, 80H
OUT CWR, AL
L2: MOV AL, 00H
OUT PORTA, AL
CALL DELAY
MOV AL, FFH
OUT PORTA, AL
CALL DELAY.
SJMP L2
    
```

DELAY: MOV CX, FFFFH

L1: DEC CX

JNZ L1

RET.

LCD INTERFACING WITH 8086:

pins:

1. V_{ss} → connect to ground.

2. V_{cc} → supply pin (connect to 5V power supply)

3. V_{ee} → connect to potentiometer.

(for contrast adjustment)

4. R_S → Register select pin.

i). Command Register [if $R_S=0$]

ii). Data Register [if $R_S=1$]

5. R/W → Read (or) write operation.

if 1

if 0 means write.

6. EN → Enable

7-14. D_0-D_7 → 8 bit data lines.

Commands

Hex. Code

Description.

01 — clear display

02 — Return ~~end~~ home

04 — decrement cursor.

06 — increment cursor.

05 — shift display right

07 — shift display left.

08 — display of cursor OFF

0A — Display OF cursor ON.

0C — Display ON cursor OFF

0E — Display ON cursor blinking.

10 - shift cursor position to left.

14 - shift cursor position to right.

18 - shift entire display to left.

1C - shift entire display to right.

80 - Force cursor to the beginning of 1st line.

81 - Force cursor to the beginning of 2nd line.

38 - two lines 5x7 matrix.

Procedure:

1. write command to LCD.

→ Give 1 to 0 transition on pin E of LCD for about 40 μ sec to 1.8msec.

2. Repeat for more command.

3. Write data.

→ Give 1 to 0 transition on pin E of LCD. Repeat for more commands.

* to connect data lines of LCD we use Port A, Port A $\rightarrow$ O/P.

* RS, R/W, EN for this we use Port C, Port C $\rightarrow$ O/P.

Program:

01 H → clear LCD

02 H → set cursor to home.

MOV AL, 80 H

OUT CWR, AL

MOV AL, 01 H

CALL COMMAND.

MOV AL, 02 H

CALL COMMAND.

MOV SI, #500

MOV CL, 09 H

L2: MOV AL, [SI]

CALL DATA

INC SI

DEC CL

JNE L2.

LCD initialization.

COMMAND: OUT PORTA, AL

MOV AL, 04 H

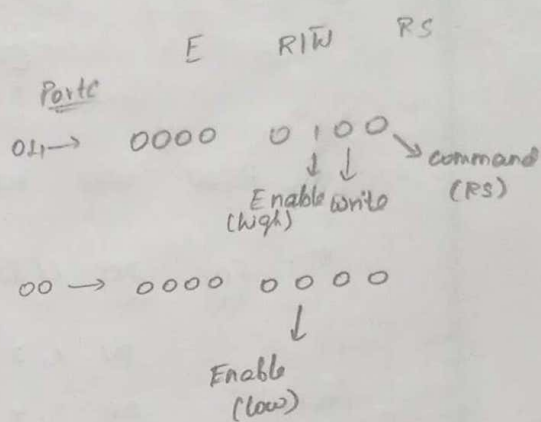
OUT PORTC, AL

MOV AL, 00 H

OUT PORTC, AL

CALL DELAY

RET.



1 to 0 transition.

DELAY: MOV CX, FFFFH

L1: DEC CX

JNZ L1

RET

DATA: OUT PA, AL

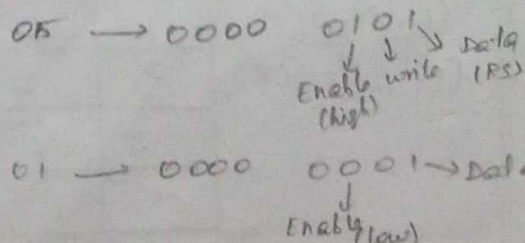
MOV AL, 05 H

OUT PC, AL

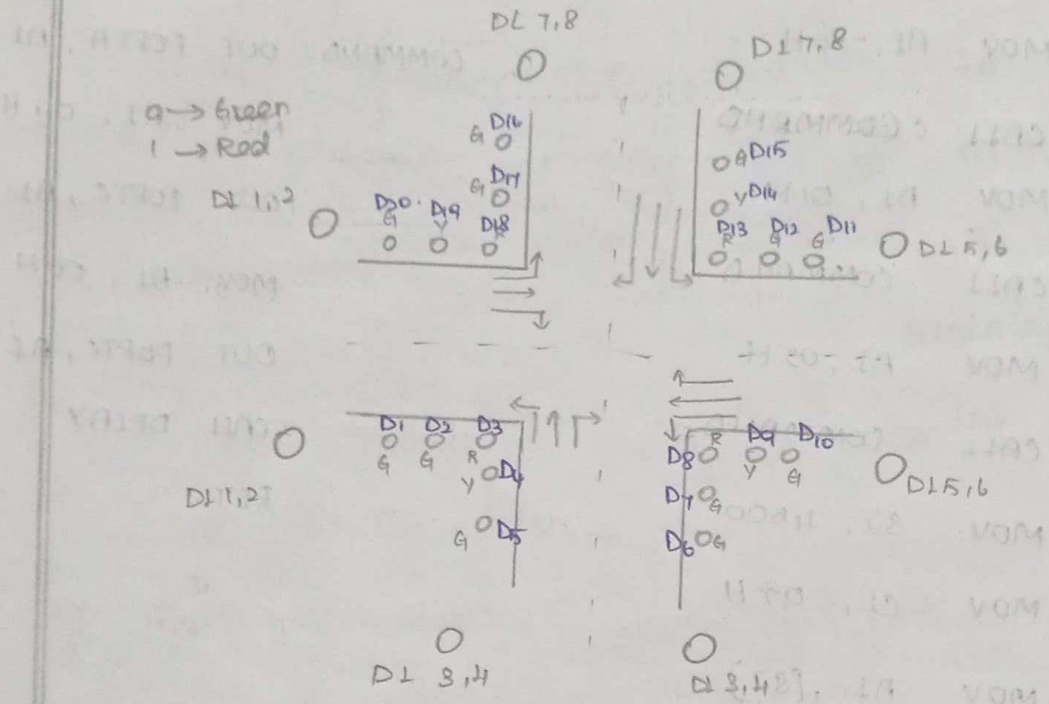
MOV AL, 01 H

OUT PC, AL

CALL DELAY.



TRAFFIC LIGHT CONTROL SYSTEM USING 8086.



*. Here we use 28 LED'S

*. For 20 LED - 20 pins.

DL 1,2 - 1 pin
DL 3,4 - 1 pin
DL 5,6 - 1 pin
DL 7,8 - 1 pin
4 pins.

∴ It has 2 pins connect, 3 ports are fully used.

Port A

(D1-D8)

7 D8	6 D7	5 D6	4 D5	3 D4	2 D3	1 D2	0 D1	
TOP	0	0	0	1	1	0	0	8C
Y	0	0	1	0	0	1	1	93
TO	1	1	0	1	0	0	0	64
Y	0	0	0	0	1	0	0	84

Port B

(D17-D20)

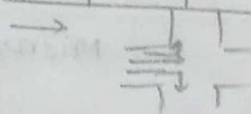
(D11,2 - D17,8)

D20	D19	D18	D17	DL 7,8	DL 5,6	DL 3,4	DL 1,2	
7	6	5	4	3	2	1	0	
1	0	0	1	1	1	1	0	9E
0	0	1	0	1	1	0	1	2D
0	0	1	0	1	0	1	1	2B
0	1	0	0	0	1	1	1	67

Port C
(D16-D9)

7	6	5	4	3	2	1	0	
1	0	0	1	0	0	0	0	90
0	0	0	1	0	0	0	1	11
0	0	1	1	0	0	1	0	32
1	0	0	1	1	0	0	0	4C

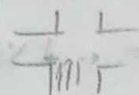
1st row



① Initialize 310 port.

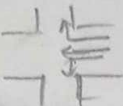
all ports are o/p.

2nd row



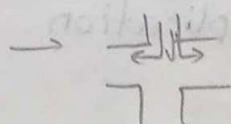
⑤ pass 1st i/p to all ports, and give delay

3rd row



④ pass 2nd i/p to all ports, and give delay

4th row



PROGRAM:

MOV CL, 04

MOV AL, 80H

OUT CWR, AL

8255 initialization.

L2: MOV SI, 1200

L1: MOV AL, [SI]

OUT PORT A, AL

INC SI

MOV AL, [SI]

OUT PORT B, AL

INC SI

MOV AL, [SI]

OUT PORT C, AL

CALL DELAY

INC SI

DEC CL

JNZ L1

JMP L2

1200	8C
1201	9E
1202	90
1203	93
1204	2D
1205	11
1206	64
1207	2B
1208	32
1209	84
120A	67
120B	4C